



Self-Challenging Language Model Agents

<https://arxiv.org/abs/2506.01716>

Research Proposal

Автор: Полудняков Владимир

Суть метода

Авторы предлагают подход, при котором модель обучается на синтетических данных, которые сама же для себя и генерирует. Процесс строится на двух ролях:

1. *Challenger*: Агент взаимодействует с инструментами и придумывает задачи в специальном формате Code-as-Task. Формат включает: текстовую инструкцию, функцию верификации (в виде кода), пример решения и заведомо ошибочные кейсы. Это позволяет автоматически тестировать и отсеивать некачественные задачи.
2. *Executor*: Агент решает сгенерированные задачи и дообучается с помощью обучения с подкреплением, используя написанную ранее функцию верификации в качестве функции вознаграждения.

Ключевые задачи, которые решает статья

1. Автоматизация обучения агентов: Устранение необходимости в дорогостоящем ручном труде, так как людям не нужно придумывать задачи, писать критерии оценки и размечать данные.
2. Улучшение навыков работы с инструментами: Развитие способности агента решать сложные многошаговые задачи с использованием внешних API и инструментов.
3. Дистилляция и самосовершенствование: Построение пайплайна, где сильная модель может обучать слабую без готовых датасетов, либо модель может улучшать саму себя автономно.

Сильные стороны

1. Элегантный контроль качества: Концепция Code-as-Task решает главную проблему синтетических данных — галлюцинации и мусор. Если сгенерированная задача не проходит собственные автотесты, она просто отбрасывается.
2. Полная автономность: Для обучения не нужны внешние данные.
3. Впечатляющая эффективность: Авторы добились более чем двукратного прироста производительности у модели Llama-3.1-8B-Instruct на сложных бенчмарках.

Слабые стороны

1. Высокий порог вхождения: Чтобы метод работал, базовая модель уже должна хорошо программировать и писать логичные тесты. Откровенно слабая модель не сможет создать для себя качественный вызов.

2. Остается доля задач, которые формально проходят фильтр, но семантически не соответствуют инструкции или реальной цели. Особенно часто это проявляется в средах, где `evaluate()` опирается на ограниченную информацию и не может отличить правильное решение в неправильном контексте
3. Потолок развития: Агент генерирует задачи только в пределах своего текущего понимания или понимания модели-учителя. Без притока данных из реального мира модель со временем может перестать обучаться принципиально новым паттернам поведения
4. Нет анализа того, как самообучение влияет на честность и безопасность

Предложение улучшения статьи

В рамках данного исследования мы фокусируемся на решении наиболее важной проблемы - качестве самопроверки агента.

Валидатор задач. После генерации CaT будем пропускать задачу через LLM, которая оценивает: соответствует ли `evaluate()` инструкции, покрывают ли `failure cases` типичные неправильные решения, нет ли очевидной невыполнимости или противоречий. Был проведен эксперимент:

1. Модель генерировала 995 задач по шаблону в статье
2. Арбитр проверял условия

Для валидации предложенного метода был проведен предварительный эксперимент на датасете из 995 задач, сгенерированных в формате CaT. Была использована асимметричная архитектура: базовая генерация задач выполнялась легковесной моделью YandexGPT Lite, а фильтрация — более мощной YandexGPT. Оценка задачи производилась по трем критериям: соответствие тестов инструкции - `eval mismatch`, покрытие типичных ошибок - `roog coverage` и логическая выполнимость - `impossible`.

Результаты показали, что было заблокировано **21.7%** сгенерированных задач. Доминирующей причиной отбраковки стала категория `roog coverage` - слабые или некорректные примеры неправильных решений, на которую пришлось **57.9%** всех невалидных примеров. Еще **42.1%** были заблокированы из-за `eval mismatch` - ошибки в `assert`-выражениях функции проверки.